University of Pretoria
EBIT Faculty
Department of Computer Science

FINAL EXAM
10 November 2025
COS341 Compiler Construction

Work-time: **3 hours**. Tasks: **5**. Points: **55**. *Average* work-time: *approximately 36 minutes per Task*

Self-made ***Crib-notes*** *are* ***allowed***. Return this task sheet together with your answer-booklet!



Assessment scheme,
as per **Study-Guide**:
Your score from this exam (out of 55) *will be added to* your semester-score (out of 45) to produce your final COS341 result (out of 100).

Internal Examiner : Prof. *S.* ***Gruner***
External Examiner: Prof. *D.* ***Coulter***, Academy of Computer Science & Software Engineering, **UJ**

Task A ..... [12 Points]

Timing-Advice: ***50 Minutes***

From your textbook you already know the the following context-free **if-else**-Grammar *is ambiguous*

ALGO	→	COMMAND
ALGO	→	COMMAND ALGO
COMMAND	→	***c***
COMMAND	→	IF ELSE
IF	→	***i*** BOOL ALGO
ELSE	→	***e*** ALGO
ELSE	→	*// nothing (nullable)*
BOOL	→	***t***
BOOL	→	***f***

A.1 [*3 Points*]
As a proof of the given grammar G's ambiguity, *provide one string* from $\boldsymbol{L}(G)$ with *two* structurally different *syntax trees*. G's **start-symbol** (and, hence, the root note of the two syntax trees) is ALGO.

A.2 [*9 Points*]
Because the given grammar G is ambiguous its *SLR parsing table* must contain *at least one* conflict. *Present* the grammar's *SLR parsing* ***table***, and ***boldly highlight*** *all* conflicts that you are able to find.

- Note: ***Only the final result*** needs to be presented for assessment.
- You *may* do the step-by-step "work-out" on some empty pages inside your Scribble-Booklet.

TURN THE PAGE ►

Need to do main translate & call-sequence.

Task C ..... [13 Points]

Timing-Advice: ***50 Minutes***

Given is the following *source-code* program, with a main algorithm and a *recursive* function, with a syntax that is *similar* to what you have already seen in our textbook:

Source Code Program	
Main	*Function*
if ((**x** > 0) **AND** (**y** > 0)) **then** { **z** := **euclid**(**x** , **y**) }	**euclid**(**a** , **b**) { **if** (**a EQ b**) **then** { **c** := **a** } **else** { **if** (**a** > **b**) **then** { **v** := (**a**−**b**) ; **w** := **b** } **if** (**a** < **b**) **then** { **v** := **a** ; **w** := (**b**−**a**) } **c** := **euclid**(**v** , **w**) } RETURN **c** } a, b, c, v, w

Moreover, also given is the following

Symbol Table :	
User-defined names	Corresponding *system-internal* names
a , b	**p1 , p2**
c	***rf***
x , y , z	**u0 , u1 , u2**
v , w	**u3 , u4**
euclid	***f***

v0 L0 v1 L1 v2 L2 v3 L3 v4 L4 v5 L5 v6 L6 v7 L7 v8 L8 v9 L9 v10 L10 v11 L11 v12 v13 v14 v15

Last but not least, also the following ***conditions*** must be taken into account:

- The *first automatically generated label* will have the name **L0**, thereafter **L1**, **L2**, etc...
- The *first automatically generated variable* will have the name **v0**, thereafter **v1**, **v2**, etc...
- The intermediate language, into which the given source program shall be translated, does *not* offer any Boolean-Logic operators such that the source program's **AND** operator must get *translated into a "chain" of* GOTO *jumps* [such as in textbook Fig.6.8].
- The source language's **EQ** gets translated to a simple =
- The source languages operator symbols > < − *are the same also in the target language.*
- The "strategy" for updating the Runtime-Stack is "Caller-saves" [as in textbook Section 9.3]
- When saving information onto the stack you need **NOT worry** about which local variable is "**live**" and which local variable is "**dead**": ***simply save all*** local variables (*without* taking the "liveness" from textbook's Chapter 8 into account here at this point).

Under all those above-mentioned conditions, use the *combined methods of Textbook Chapter 6 and Textbook Chapter 9* in order to *present the complete* ***intermediate code*** of the given source program.

- Note: ***Only the final result*** needs to be presented for assessment.
- You *may* do the step-by-step "work-out" on some empty pages inside your Scribble-Booklet.

TURN THE PAGE ►

Task D ..... [13 Points]

Timing-Advice: ***50 Minutes***

The following program is **valid C++** code which uses GOTO commands (though it is normally not recommended to use GOTO in high-level source-code). ***Line-numbers*** are provided as ***//comments*** *behind* every relevant instruction: altogether there are ***17*** relevant line numbers.

- Note: This C++ program, which contains two loops "nested" into each other, does does *not do anything particularly "useful"* – it merely computes some silly numbers just for fun **:)**
- Also note: This C++ program "simulates" Two-Address-Code (*2AC*) as it does *not* contain any command of the form $W = U \; op \; V$ with U,V,W being *three different* variables.

```
#include <iostream> // no line number
int x,y,i,j,a,b;    // no line number
int main()          // no line number
{
    x = 5 ;                       // Line  1
    y = x ;                       // Line  2
    i = y ;                       // Line  3
    a = 0 ;                       // Line  4
 L0:                              // Line  5
    j = y ;                       // Line  6
    b = 0 ;                       // Line  7
 L1:                              // Line  8
    b = b + j ;                   // Line  9
    j = j - 1 ;                   // Line 10
    if( j > 0 ) { goto L1 ; }     // Line 11
    y = y - 1 ;                   // Line 12
    b = b + i ;                   // Line 13
    a = a + b ;                   // Line 14
    i = i - 1 ;                   // Line 15
    if( i > 0 ) { goto L0 ; }     // Line 16
    return a ;}                   // Line 17
```

D.1 [*10 Points*]

Apply the methods from Textbook Chapter 8 to determine the *liveness-sets* **in**[$\mathcal{L}$] and **out**[$\mathcal{L}$] for each line $\mathcal{L}$ of the given program. Present your *final* results *neatly* in a *table with three columns*, in the following format:

Line-number $\mathcal{L}$	**out**[$\mathcal{L}$]	**in**[$\mathcal{L}$]

- Note: ***Only the final result*** must be presented for assessment.
- You can do the step-by-step "work-out" on some empty pages inside your Scribble-Booklet.

D.? [*3 Points*]

Present the ***interference graph*** for the program's six variables **x**, **y**, **i**, **j**, **a**, **b**.

- **Note**: you do not need to mark the graph's nodes in different colours. The graph's structure, *without* any colours, is enough.
- ***Only the final result*** needs to be presented for assessment.
- You *may* do the step-by-step "work-out" on some empty pages inside your Scribble-Booklet.

Task E ..... [5 Points]

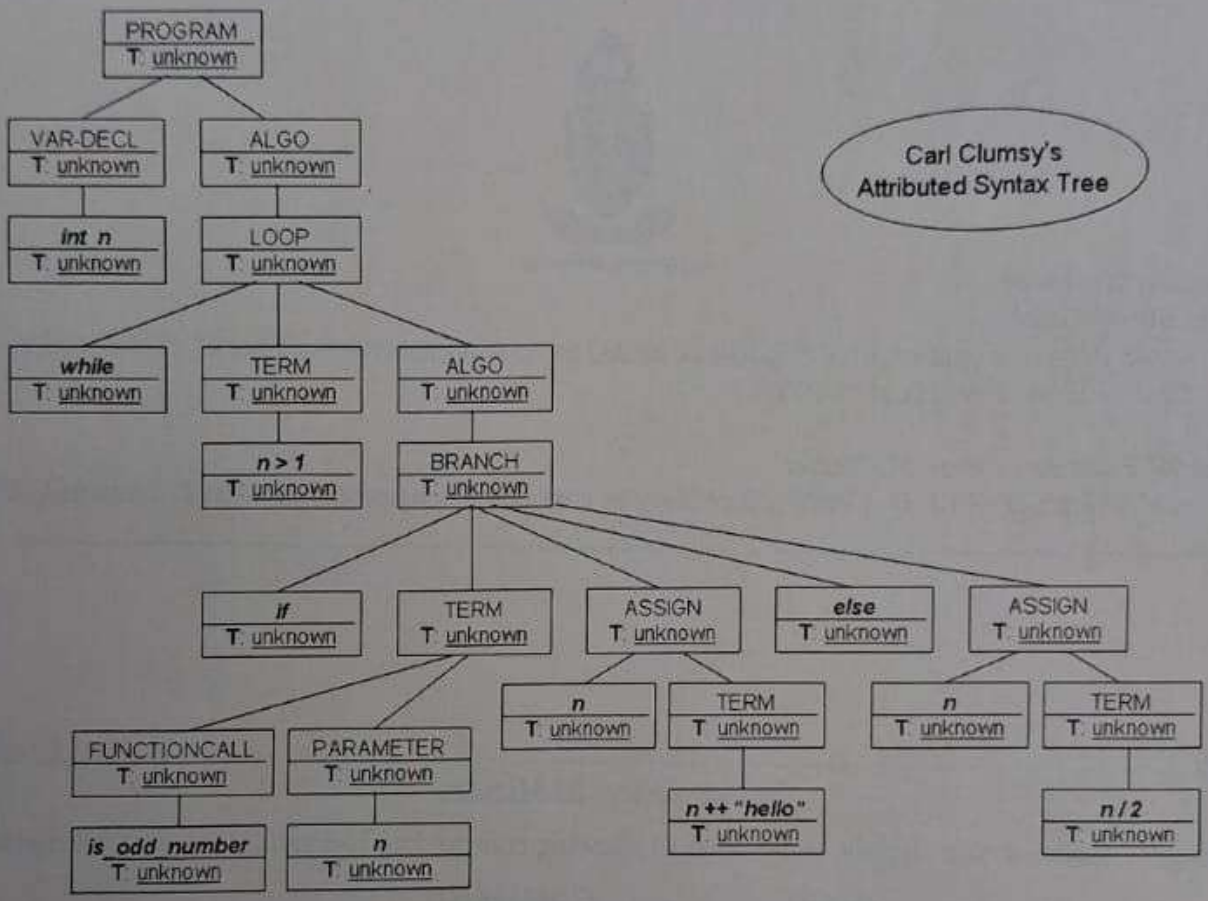
Timing-Advice: ***10 Minutes***

In *simple* semi-English *Pseudo-Code* present the *few main steps* which a "greedy" *Largest-Fit-First* (LFF) *algorithm* must carry out in order to *finally translate* machine-*independent intermediate* code into *machine-specific* target code for a particular hardware-platform.

Task B ..... [12 Points]

Timing-Advice: ***20 Minutes***

Carl Clumsy has prepared the following *attributed* syntax tree for *type checking*. Initially, *before* the type checker has been invoked, every node of the tree contains the additionally attached *Type Attribute* **T**: unknown. Thereby you may assume that all nodes of the syntax tree belong to the *same name scope* – in other words: the variable ***n***, which you can see repeatedly throughout the tree, is *everywhere the same* semantic object.



While the type checker is working by "crawling up and down" the syntax tree, it will step-by-step ***replace*** every unknown **T**-Attribute in every node by *one* of the following *decided* **T**-Attributes:

- numeric
- boolean
- string
- correct
- wrong

Present the attributed syntax tree, with all the new **T**-Attributes on all its nodes, as it will look like after the type checker has entirely *completed* its semantic analysis.

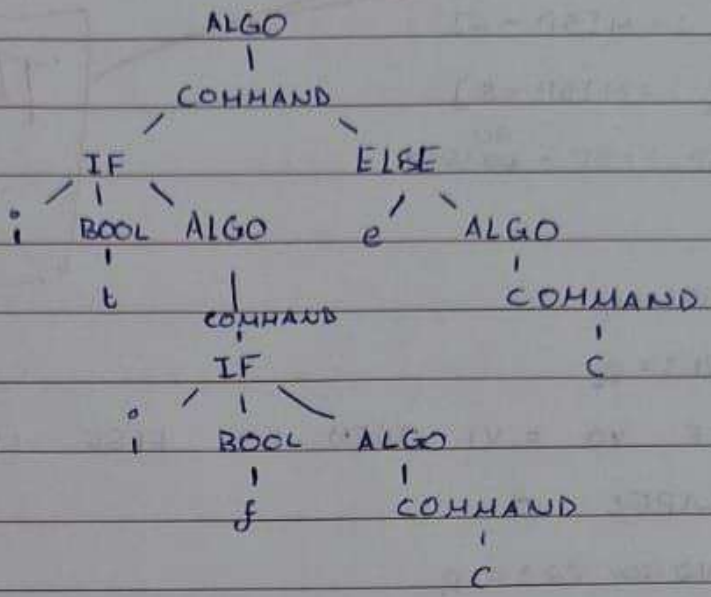
- Note: ***Only the final result*** needs to be presented for assessment.
- You *may* do the step-by-step "work-out" on some empty pages inside your Scribble-Booklet.

TURN THE PAGE ►

Task A

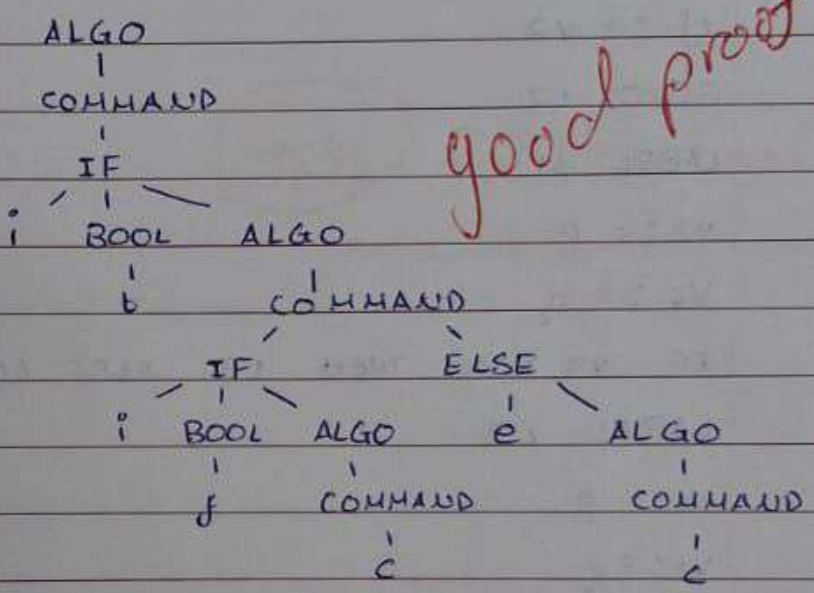
A.1 i b i f c e c

good string ✓



✓

A.2



good proof ✓

3/3

X

X — [Sub-task A.2] — X

not done.

X

Task D

10/10

D.1

Line number	Out [L]	In [L]
1	x	∅
2	y	x
3	y,i	y
4	y,i,a	y,i
5	y,i,a	y,i,a
6	j,y,i,a	y,i,a
7	b,j,y,i,a	j,y,i,a
8	b,j,y,i,a	b,j,y,i,a
9	j,y,b,i,a	b,j,y,i,a
10	j,y,b,i,a	j,y,b,i,a
11	y,b,i,a,j	j,y,b,i,a
12	b,i,a,y	y,b,i,a
13	a,b,i,y	b,i,a,y
14	i,y,a	a,b,i,y
15	i,y,a	i,y,a
16	y,i,a	i,y,a
17	∅ ✓	a ✓

Nice

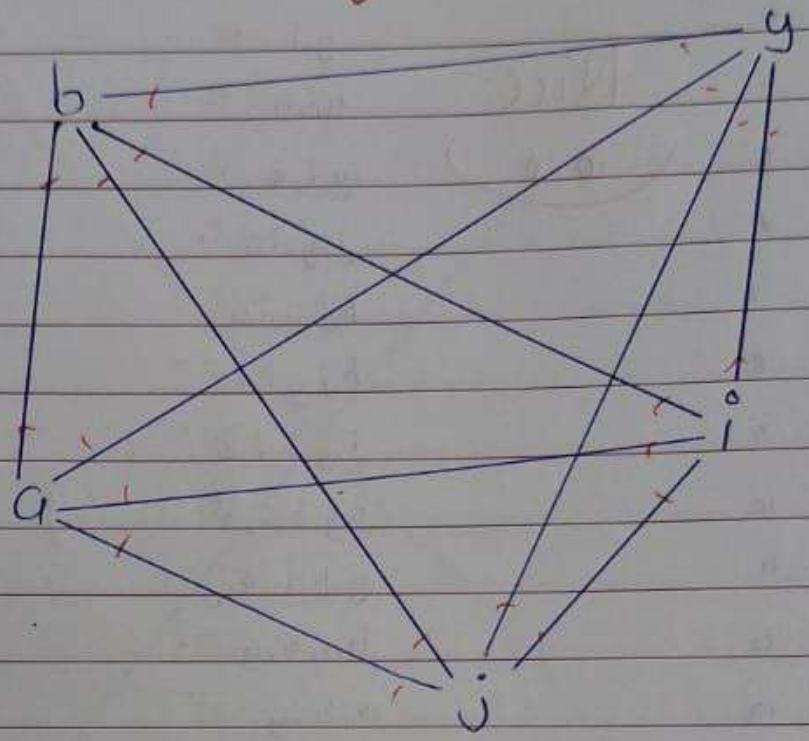
! D.2 on next page

Task D

P. 2

3/3

x ✓



Task E

We call a list of of intermediatory ~~patterns~~ paired with their replacement machine code. We recieve the list in a ~~manner~~ manner that lists the ~~longest~~ patterns first, therefore

Loop

Trans$_{IR}$ (ID) = case IR of

long pattern : • replacement machine code

(2)

next longest pattern : replacement machine code with similar prefixes to previous pattern

For Memo, see our Lecture Slides (Ch. 7)

```
V12 := V13 - V14
U4 := V12
LABEL L6

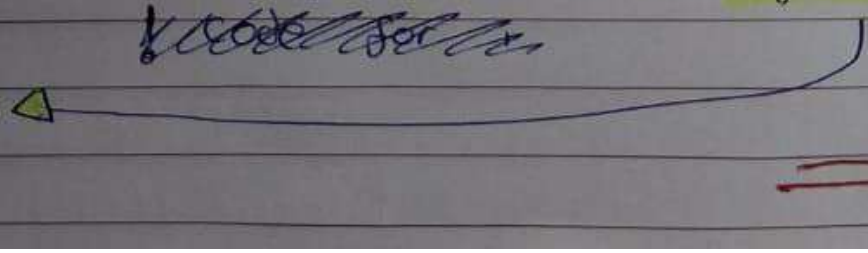
M[SP] := P1
M[SP+4] := P2
M[SP+8] := rf
M[SP+12] := U3
M[SP+16] := U4
SP := SP - #12
M[SP+4] := P1
M[SP+8] := P2
M[SP] := L7
GOTO f
LABEL L7
rf := M[SP+4]
SP := SP + #12
P1 := M[SP]
P2 := M[SP+4]
rf := M[SP+8]
U3 := M[SP+12]
U4 := M[SP+16]

LABEL L2
SP := SP + 20
M[SP+4] := rf
GOTO M[SP]
```

(4,4) for the Euclid ...

Noted ✓

! The resto of the code is after Task B. It should sit here



Please page back to see the code

Task C - Combined

This is the code for the MAIN Function

```
V15 := U0
V16 := 0
IF V15 > 0 THEN L10 ELSE L9
LABEL L10
V17 := U1
V18 := 0
IF V17 > V18 THEN L8 ELSE L9
LABEL L8
M[SP] := U0
M[SP+4] := U1
M[SP+8] := U2
SP := SP - 12
M[SP+4] := U0
M[SP+8] := U1
M[SP] := return address L11
GOTO f
LABEL L11
U2 := M[SP+4]
SP := SP + 12
U0 := M[SP]
U1 := M[SP+4]
U2 := M[SP+8]
LABEL L9
```

+ (4,3) for the MAIN

TASK C

~~CODE FOR MAIN PROG~~ ~~It should be inserted here~~

This is the Code for the Euclid Function

LABEL f

$P_1 := M[SP + 4]$

$P_2 := M[SP + 8]$

$SP := SP - 20$

$V0 := P_1$

$V1 := P_2$

IF V0 = V1 THEN L0 ELSE L1

LABEL L0

V2 := P_1

rf := V2

GOTO L2

LABEL L1

V3 := P_1

V4 := P_2

IF V3 > V4 THEN L3 ELSE L4

LABEL L3

V6 := P_1

V7 := P_2

V5 := V6 - V7

U3 := V5

V8 := P_2

U4 := V8

LABEL L4

V9 := P_1

V10 := P_2

IF V9 < V10 THEN L5 ELSE L6

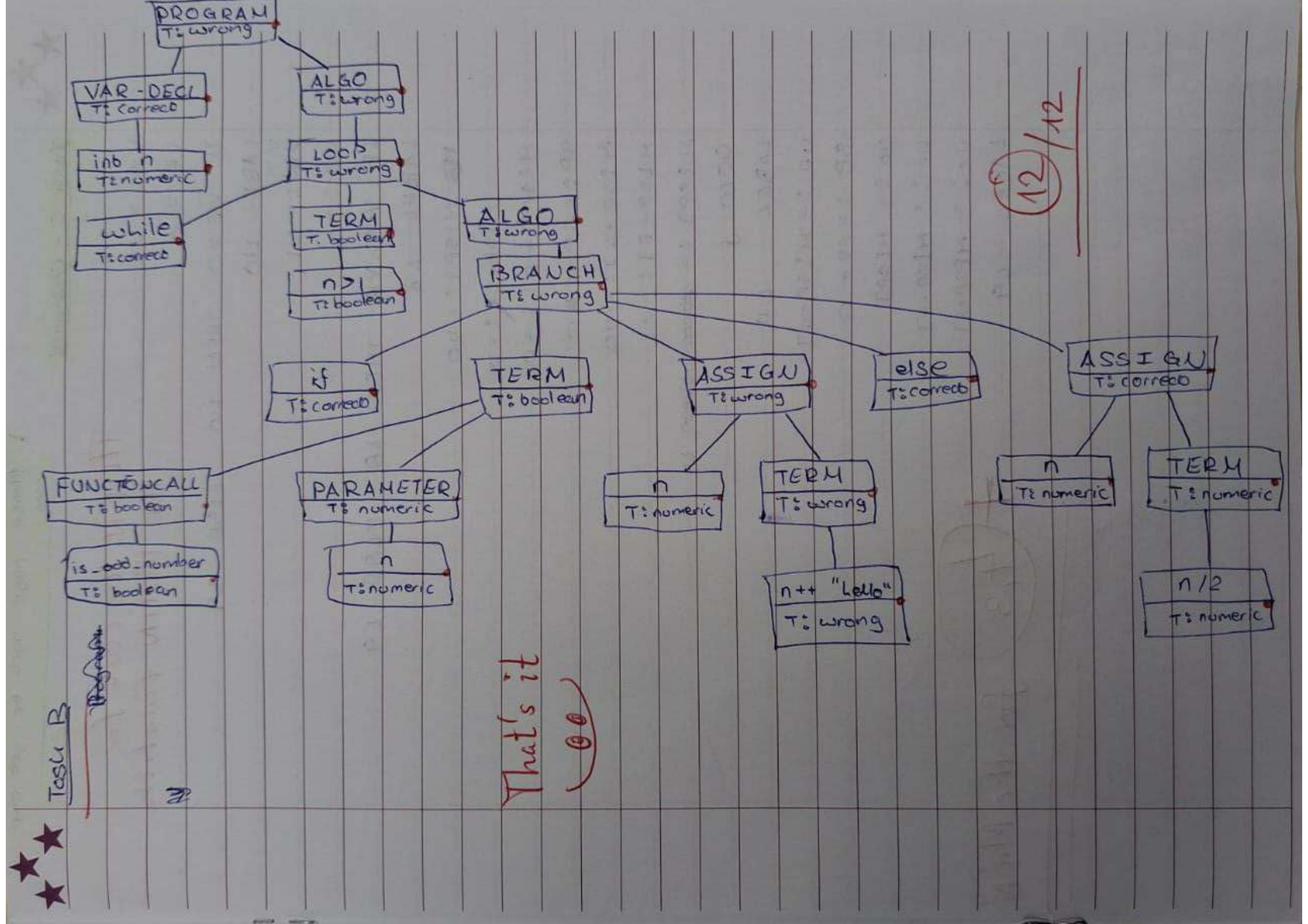
LABEL L5

V11 := P_1

U3 := V11

V13 := P_2

V14 := P_1


PROGRAM
T: wrong
VAR-DECL
T: correct
int n
T: numeric
while
T: correct
ALGO
T: wrong
LOOP
T: wrong
TERM
T: boolean
n>1
T: boolean
ALGO
T: wrong
BRANCH
T: wrong
if
T: correct
TERM
T: boolean
ASSIGN
T: wrong
else
T: correct
ASSIGN
T: correct
FUNCTIONCALL
T: boolean
is_odd_number
T: boolean
PARAMETER
T: numeric
n
T: numeric
n
T: numeric
TERM
T: wrong
n++ "Hello"
T: wrong
n
T: numeric
TERM
T: numeric
n/2
T: numeric
12/12
That's it
Task B